Programación Concurrente y Distribuida

Tema 8. Invocación Remota

Contexto

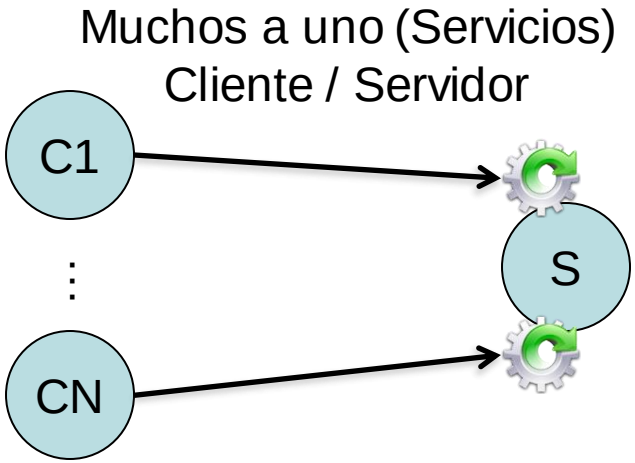
- Sistemas distribuidos:
  - No hay una memoria central compartida por todos
- Problemas
  - Comunicación
  - Sincronización
  - Exclusión mutua
- Soluciones
  - Mecanismos de paso de mensajes síncrono y asíncrono
- Inconvenientes
  - Bajo nivel

Invocación Remota

- Mecanismo de alto nivel basado en paso de mensajes.
- Extensión de la abstracción operacional a entornos distribuidos.
- Carácterísticas
  - Direccionamiento: Asimétrico (Cliente/Servidor)
  - Capacidad: nula
  - Flujo: bidireccional
  - Sincronización temporal: redezvous extendido

Funcionamiento

- Cada cliente indica su interés por que se ejecute algún servicio, pero es el servidor quien lo ejecuta iniciando la invocación.
- El servidor lo hace cuando quiere y sólo si hay algún cliente preparado en ese momento.
- Si varios clientes quieren ejecutar servicios del mismo servidor sólo se atiende a uno, los demás esperan su turno.
- Lo servidores publican los puntos de entrada que ofrecen.



Invocación Remota

- Operaciones
  - Invocación:

```
id_destino.id_procedimiento(argumentos);
```

  - Punto de entrada:

```
accept id_procedimiento(parámetros) {
  // código
  return resultado;
}
```

- Sincronización temporal: Rendezvous Extendido
  - Emisor y receptor quedan bloqueados hasta producirse sincronización temporal. Después, el emisor queda bloqueado mientras receptor ejecuta el procedimiento. Finalmente, el emisor recibe el resultado y ambos continúan.

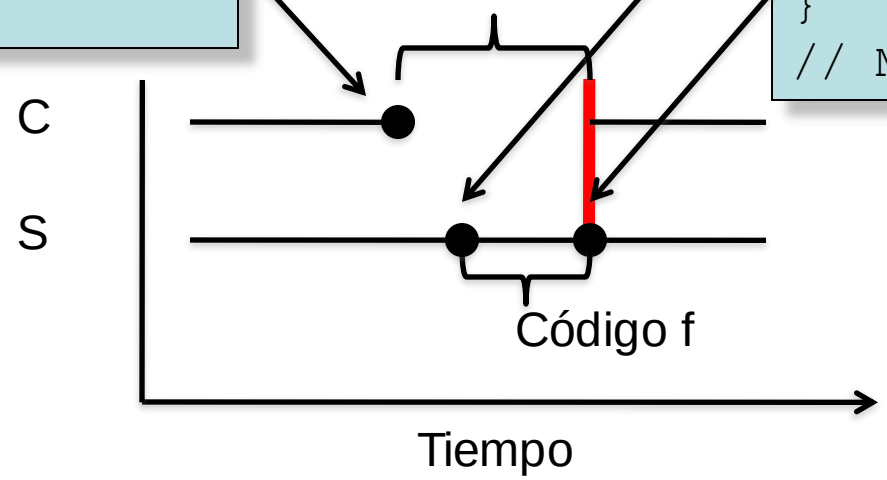
Rendezvous Extendido 1/2

C (Cliente)

```
// Sentencias
r = S.f(argumentos);
// Más sentencias
```

S (Servidor)

```
// Sentencias
accept f(parametros) {
  // Código de f
  return resultado;
}
// Más sentencias
```



- El emisor se bloquea pero el receptor no porque el emisor ya estaba listo. Cuando el receptor termina de ejecutar el código de **f**, y **C** recibe el resultado, ambos continúan.

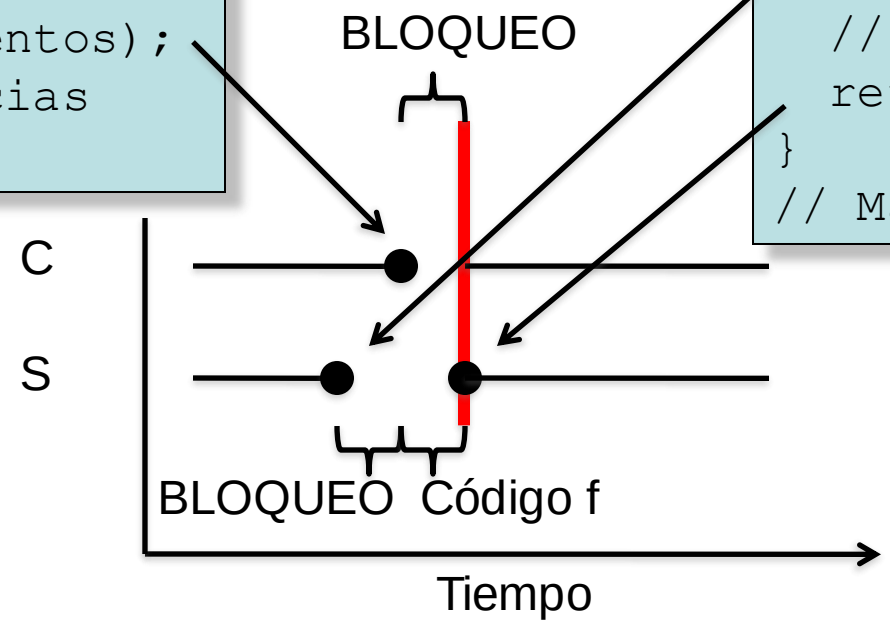
Redezvous Extendido 2/2

C (Cliente)

```
// Sentencias
r = S.f(argumentos);
// Más sentencias
```

S (Servidor)

```
// Sentencias
accept f(parametros) {
  // Código de f
  return resultado;
}
// Más sentencias
```



- El receptor se bloquea hasta que el emisor hace la invocación. El emisor se bloquea mientras el receptor lleva a cabo el código de **f**. Tras enviar/recibir el resultado ambos continúan.

Espera Selectiva

- Es posible, y deseable, utilizar espera selectiva para que un mismo servidor pueda ofrecer distintos servicios sin condicionar el orden en que se llevarán a cabo.

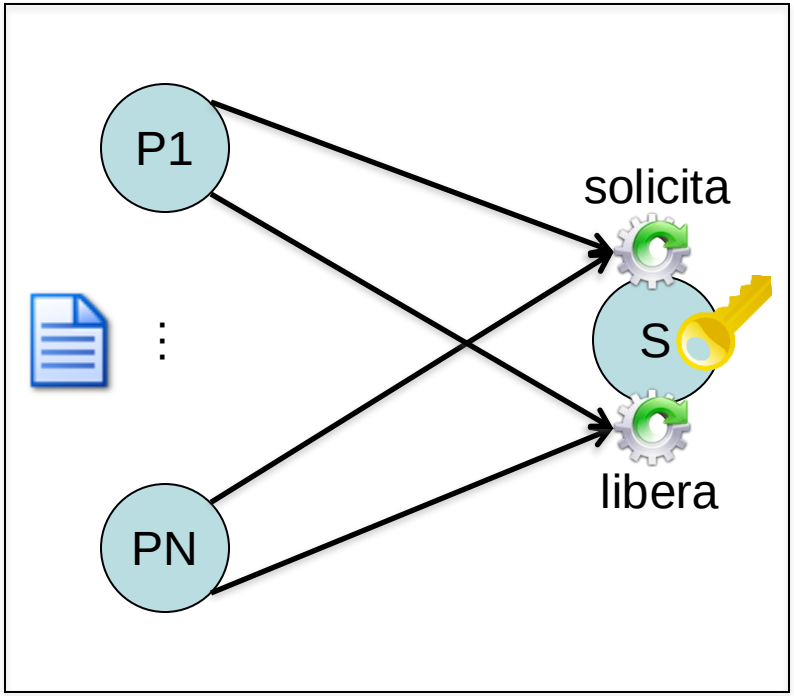
No se puede usar el valor de los parámetros en la condición del **`when`**

```
select {
    when condicion1 =>
    accept f1(parámetros) {
      ...
    }
  or
    ...
  or
    when condicionN =>
    accept fN(parámetros) {
      ...
    }
}
```

El problema de la Exclusión Mutua

- Al no existir una memoria compartida no hay necesidad de garantizar la exclusión mutua al acceder a ninguna variable.
- ¿Qué pasa con los recursos hardware o software centralizados?
- **Ejemplo de problema**: Garantizar el acceso en exclusión mutua a un fichero cuando N procesos quieren utilizarlo.

Solución con Invocación Remota



Pi

```
while (true) {
  S.solicita();
  // Sección Crítica
  S.libera();
}
```

S

```
boolean ocupado = false;
while (true) {
  select {
      when !ocupado =>
      accept solicita() {
        ocupado = true;
      }
    or
      accept libera() {
        ocupado = false;
      }
  }
}
```

Ejercicios propuesto:

1)Simular semáforo general

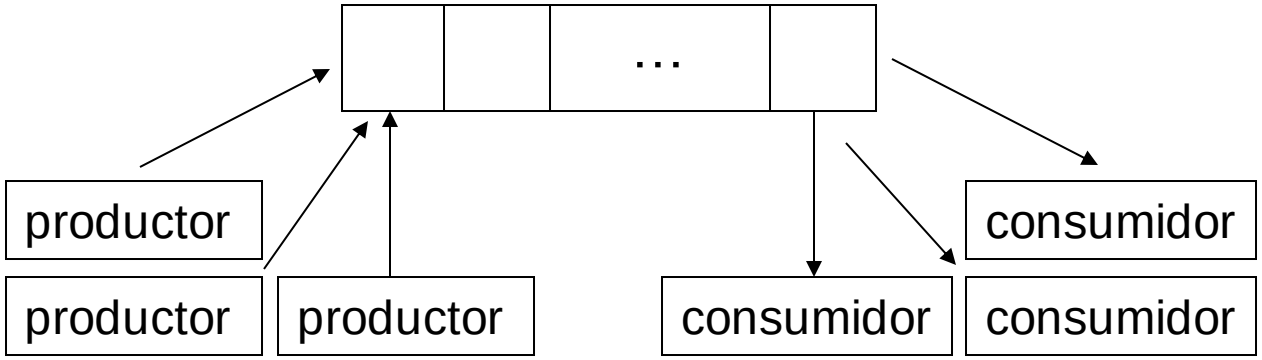
2)¿Se podría hacer sin select?

Problemas Clásicos

- Productor/Consumidor
- Lectores y Escritores
- Cena de los Filósofos

Productor**es** / Consumidor**es**

- Procesos que generan información (productores) que es utilizada por otros procesos (consumidores).
- Posible diferencia de velocidad de ejecución requiere **algún tipo de buffer** para desacoplar velocidades.
- Necesidad de acceder en exclusión mutua al buffer.

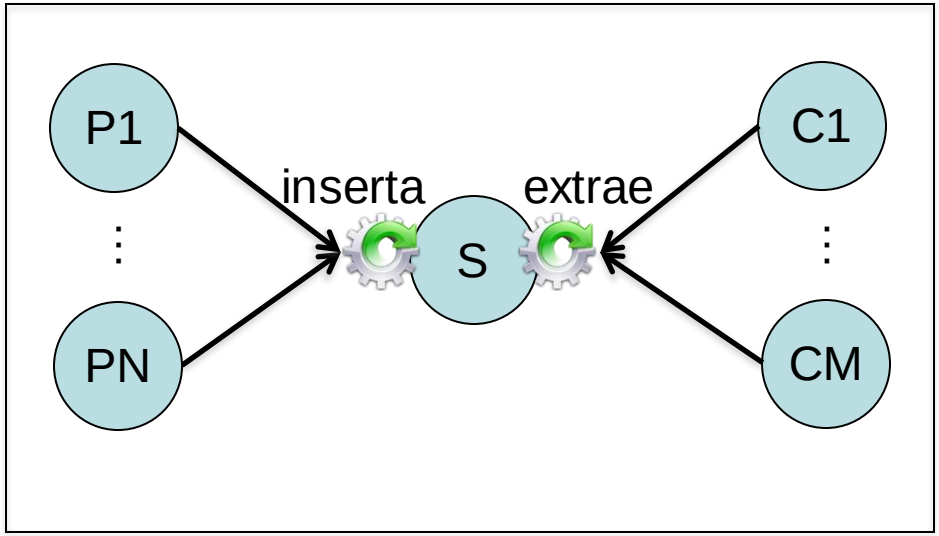


Análisis Buffer Limitado (I.R.)

- Recursos no compartibles
  - Cola de tamaño T
  - Se insertan y se extraen elementos
- Condiciones de sincronización
  - Sólo se inserta si hay espacio
  - Sólo se extrae si hay elementos

- Cola será una variable de un proceso controlador
- La inserción será un servicio ofrecido por el controlador con el elemento como parámetro.
- La extracción será un servicio ofrecido por el controlador que devuelva el elemento.
- El Controlador tendrá que decidir cuando admitir inserciones y cuando responder a extracciones

Productores/Consumidores (I.R.)



Pi

```
while (true) {
  // Producir
  Elemento e = new Elemento();
  // Insertar
  S.inserta(e);
}
```

Ci

```
while (true) {
  // Extraer
  Elemento e = S.extrae();
  // Consumir
  e.usa();
}
```

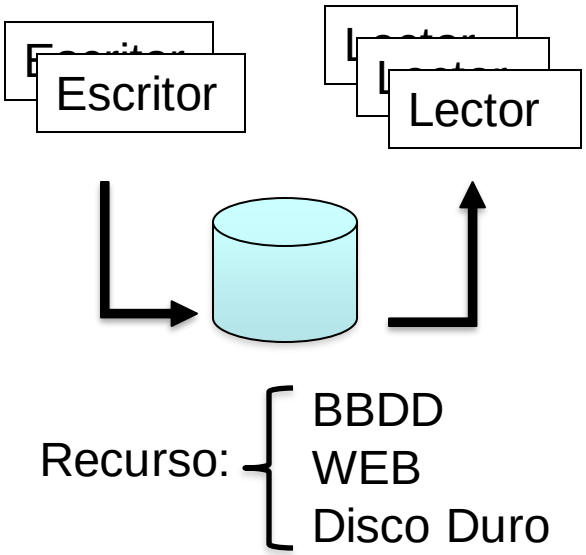
Productores/Consumidores (I.R)

S

```
Cola cola = new Cola(T);
while (true) {
  select {
      when !cola.llena() =>
      accept inserta(Elemento e) {
        cola.push(e);
      }
    or
      when !cola.vacia() =>
      accept extrae() {
        return cola.pop();
      }
  }
}
```

Lectores / Escritores

- Procesos que acceden a un recurso en modo lectura (lectores) y procesos que acceden en modo escritura al mismo recurso (escritores).



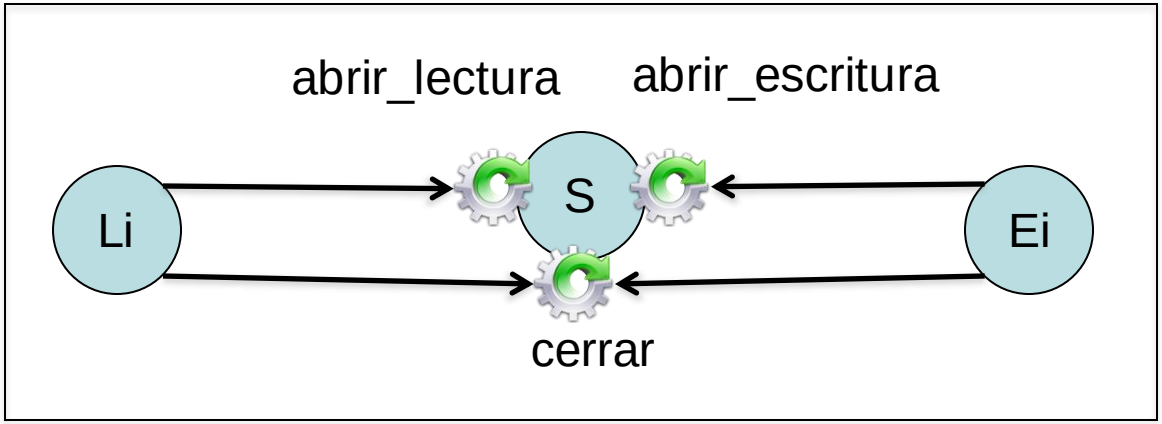
- **Prioridad lectores:** ningún lector debe esperar salvo que un escritor esté escribiendo. Un lector no espera por el simple hecho de que haya un escritor esperando.
- **Prioridad escritores:** cuando un escritor espera, debe escribir lo antes posible. Es decir, ningún nuevo lector debe iniciar su lectura

Análisis L/E prioridad L (I.R.)

- Recursos no compartibles
  - nl y escribiendo
  - RECURSO sólo a veces
- Condiciones de sincronización
  - leer si !escribiendo
  - escribir si !escribiendo y nl==0

- Necesitamos mantener el valor de **nl** y **escribiendo** en un proceso controlador
- El controlador recibirá peticiones de lectura o escritura desde los distintos procesos (dos servicios)
- Cada proceso deberá bloquearse hasta que le concedan permiso (I.R. es bloqueante)
- Finalmente habrá que informar al controlador de la liberación del recurso (otro servicio parametrizado)

Lectores/Escritores prioridad Lectores (I.R.)



Lector i

```
while (true) {
  S.abrir_lectura()
  // Leer
  S.cerrar('L');
}
```

Escritor i

```
while (true) {
  S.abrir_escritura();
  // Escribir
  S.cerrar('E');
}
```

Lectores/Escritores prioridad Lectores (I.R.)

s

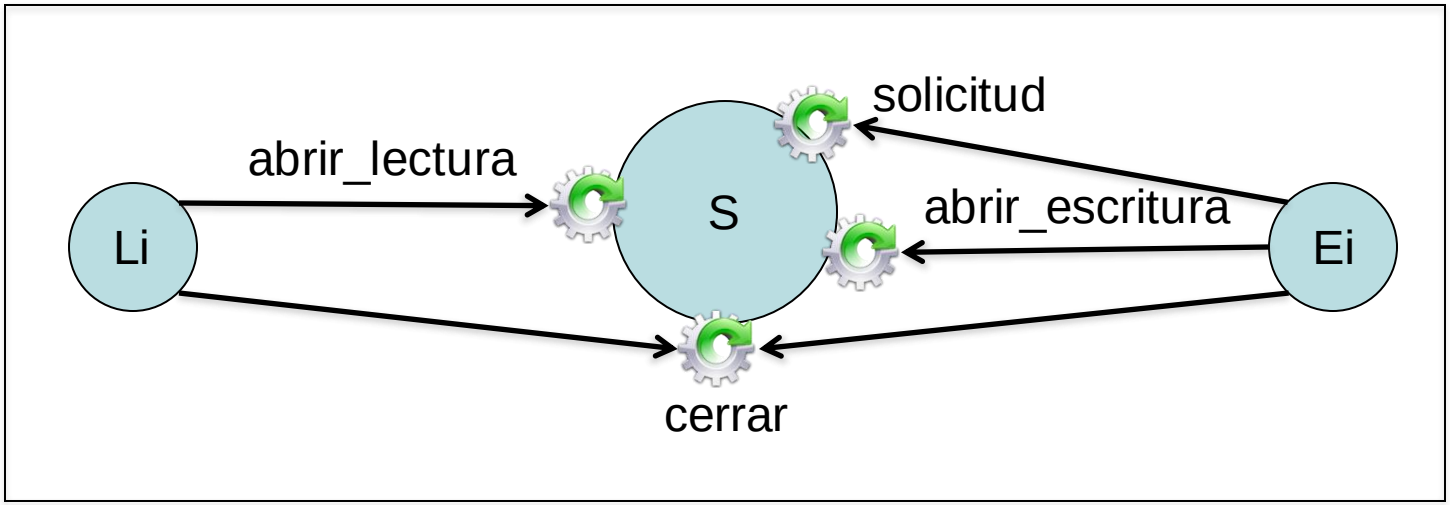
```
int nl = 0;
boolean escribiendo = false;
while (true) {
  select {
      when !escribiendo =>
      accept abrir_lectura()
        {  nl++;  }
    or
      when (nl==0) && !escribiendo =>
      accept abrir_escritura()
        {  escribiendo = true; }
    or
      accept cerrar(char msg) {
        if (msg == 'L') nl--;
        if (msg == 'E') escribiendo = false;
      }
  }
}
```

Análisis L/E prioridad E (I.R.)

- Recursos no compartibles
  - nl, nee y escribiendo
  - RECURSO sólo a veces
- Condiciones de sincronización
  - leer si !escribiendo y nee == 0
  - escribir si !escribiendo y nl==0

- Necesitamos mantener el valor de **nl**, **nee** y **escribiendo** en un proceso controlador
- El controlador recibirá peticiones de lectura o escritura desde los distintos procesos (dos servicios)
- Para saber si hay peticiones de escritura pendientes usamos otro servicio de solicitudes de escritura.
- Cada proceso deberá bloquearse hasta que le concedan permiso (I.R. es bloqueante)
- Finalmente habrá que informar al controlador de la liberación del recurso (otro servicio parametrizado)

Lectores/Escritores prioridad Escritores (I.R.)



Lector i

```
while (true) {
  S.abrir_lectura();
  // Leer
  S.cerrar('L');
}
```

Escritor i

```
while (true) {
  S.solicitud('E');
  S.abrir_escritura();
  // Escribir
  S.cerrar('E');
}
```

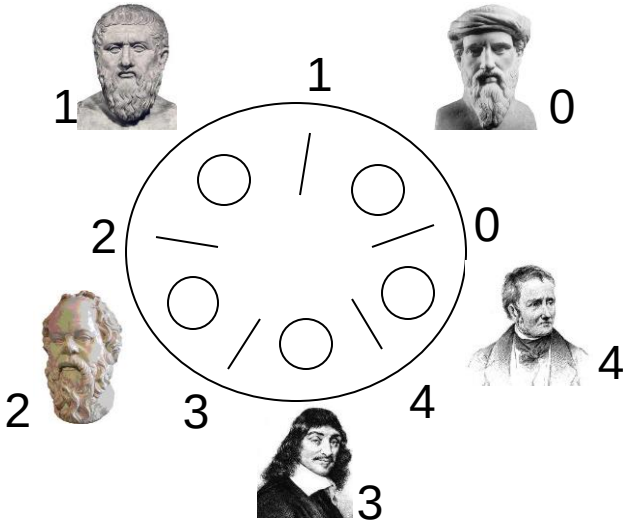
Lectores/Escritores prioridad Escritores (I.R.)

S

```
int nl = 0; int nee = 0;
boolean escribiendo = false;
while (true) {
  select {
      accept solicitud(char msg)
        {  if (msg == 'E') nee++;  }
    or
      when (nee == 0) && !escribiendo =>
      accept abrir_lectura()
        {  nl++;  }
    or
      when (nl==0) && !escribiendo =>
      accept abrir_escritura() {
        escribiendo = true;
        nee--;
      }
    or
      accept cerrar(char msg) {
        if (msg == "L") nl--;
        if (msg == "E") escribiendo = false;
      }
  }
}
```

Filósofos

- Cinco filósofos se dedican a pensar y comer de forma alterna. Para comer usan una mesa con cinco sillas, cinco palillos y cinco platos.
- Cuando un filósofo decide comer intenta coger sus dos palillos y sólo comerá cuando tenga los dos.



- Procesos que necesitan utilizar varios recursos no compartibles simultáneamente. El número de recursos no es suficiente para abastecer a todos los procesos al mismo tiempo.

Análisis Filósofos (I.R.)

- Recursos no compartibles
  - **palillos o asientos**

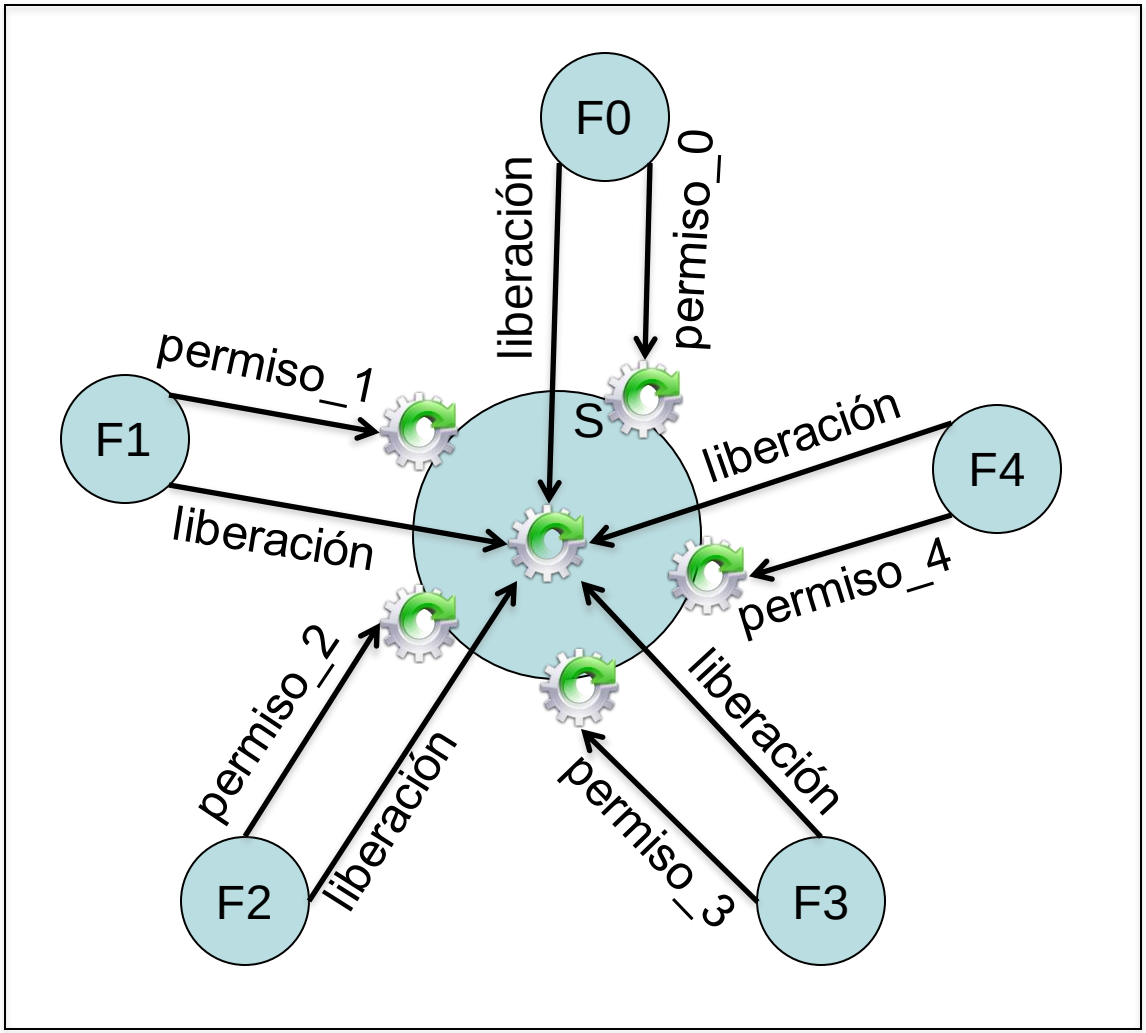
- **estado de los palillos o de los filósofos** como variables del un proceso controlador

- Condiciones de sincronización
  - Filósofo **i** sólo come si sus dos palillos están libres

- Habrá un servicio de petición por filósofo, y uno para informar de liberación parametrizado.
- Las peticiones sólo se atienden si se cumplen todas las condiciones necesarias que dependen del filósofo que se quiere sentar

Filósofos (I.R.) V1.0

V1



Filósofo i

```
while (true) {
  S.permiso_i();
  // Comer
  S.liberación(i);
  // Pensar
}
```

Filósofos (I.R.) V1.0

s

```
boolean libre[] = {true,true,true,true,true};
while (true) {
  select {
      when libre[4] && libre[1] =>
      accept permiso_0() {
        libre[0] = false;
      }
    or
      ...
    or
      when libre[3] && libre[0] =>
      accept permiso_4() {
        libre[4] = false;
      }
    or
      accept liberación(int idFilosofo) {
        libre[idFilosofo] = true;
      }
  }
}
```

2º Análisis Filósofos (I.R.)

- Recursos no compartibles
  - **palillos o asientos**

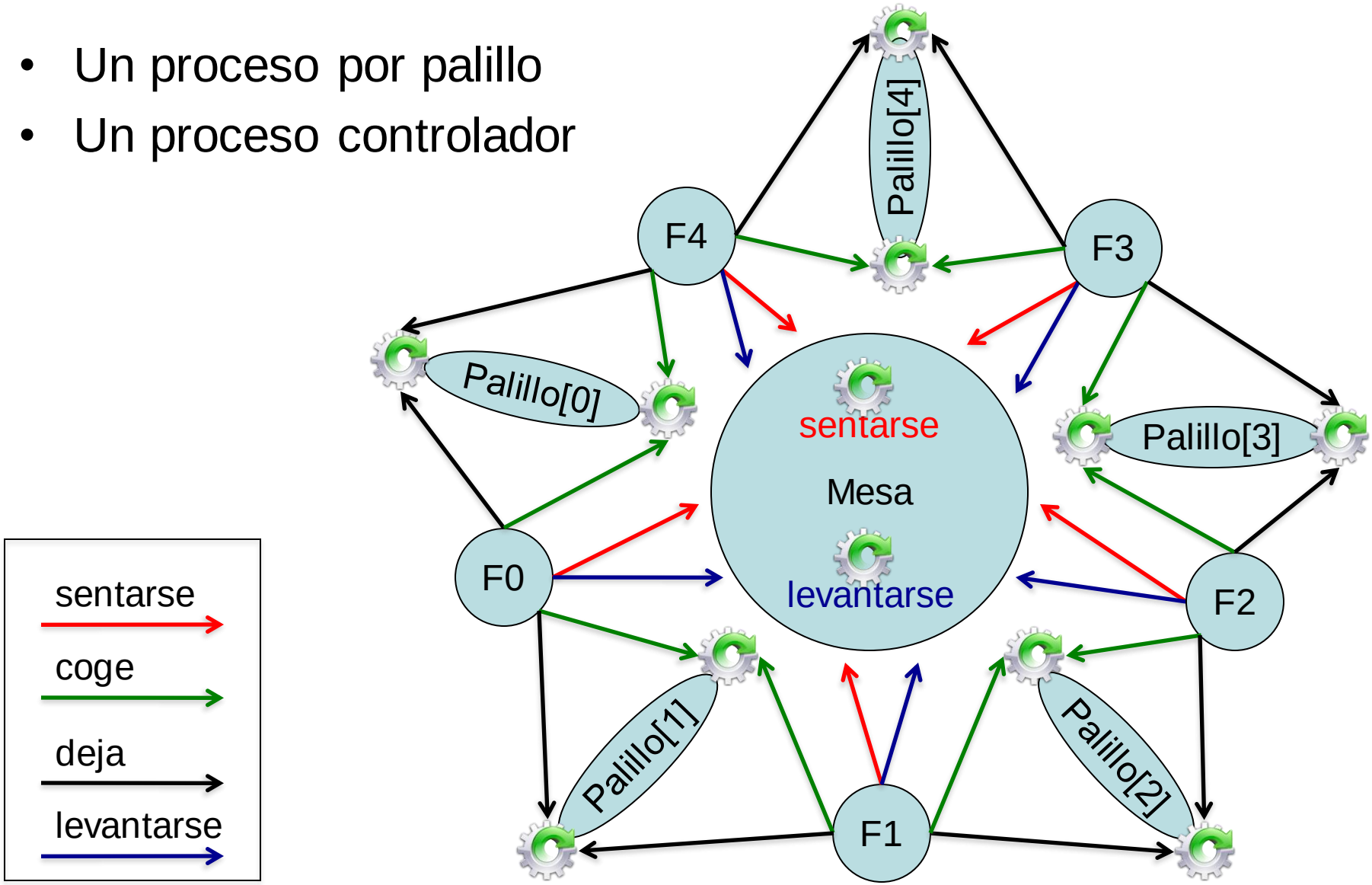
- **Cada proceso controlador de palillo ofrece dos servicios: coge y deja**

- Condiciones de sincronización
  - Filósofo **i** sólo come si sus dos palillos están libres
- Requisito extra (V2.0)
  - Cada palillo lo controla su propio proceso

- Para evitar interbloqueos se añade un proceso adicional que controla el número de filósofos que pueden sentarse a 2.
- Si coinciden dos filósofos consecutivos uno de ellos come seguro y al levantarse lo hará el otro.

Filósofos (I.R.) V2.0

- Un proceso por palillo
- Un proceso controlador



Filósofos (I.R.) V2.0

Palillo[i]

```
while (true) {
  accept coge(){};
  accept deja(){};
}
```

Mesa

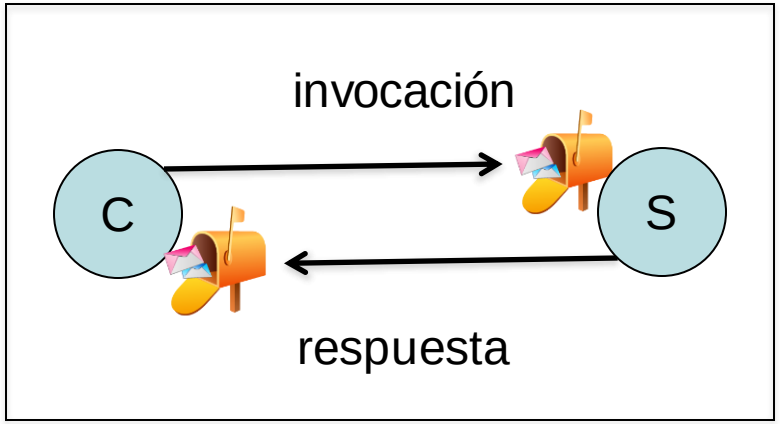
```
int libres = 2;
while (true) {
  select {
      when libres > 0 =>
      accept sentarse() {
        libres--;
      }
    or
      accept levantarse() {
        libres++;
      }

  }
}
```

Filósofo i

```
while (true) {
  Mesa.sentarse();
  Palillo[i % 5].coge();
  Palillo[i+1 % 5].coge();
  // Comer
  Palillo[i % 5].deja();
  Palillo[i+1 % 5].deja();
  Mesa.levantarse();
  // Pensar
}
```

Simulación de I.R. con Buzones

Si tenemos buzones podemos simular invocación remota sin problemas.



Cliente (C)

```
// Servidor.funcion(args);
Mailbox bRes = new Mailbox();
send(f, msg(bRes+args));
receive(bRes, resultado);
```

Servidor (S)

```
// accept funcion(params) {
//    Código
//    return resultado;
// }
receive(f, msg(bRes+params));
Resultado = funcion(params);
send(bRes, resultado);
```